안녕하세요.
Jquery 강좌 중인 Falcon 입니다.

오늘은 저번 강좌에서도 말했듯이 Jquery가 가지고 있는 이벤트 사용법을 익혀보려고 합니다. 이미 기본적인 이벤트 방법은 알려드렸습니다만, Jquey만의 이벤트 사용법이 있으니 그 부분을 한 번 알아봅시다.

.on('events' , [selector], handler(eventObject))

사실 이렇게 보면 아시는 분들도 있을테고 모르시는 분들이 더 많을 겁니다. 하지만 이 부분이 이벤트를 사용하기 위해서 여러분들이 알아야내야할 과제 입니다. 이미 Jquery 사이트에는 여러분들이 이런 부분들을 사용할 수 있는 예제를 보여주고 있으니 Jquery에서 궁금한 것이 있으면 구글링이나 API 문서를 참고하시기 바랍니다.

```
1  $(document).ready(function() {
2    $('#Jquery').on('click', function(event) {
3      alert("Jquery");
4    });
5  });
```

master.js

저번시간에 배운 이 코드는 여러분들이 프로젝트를 하면서 아마 가장 많이 쓰게 될 API 입니다. 이유는 클릭을 했을 때 이벤트를 제어해야하기 때문입니다. 여기서 사용된 것은 위에서 events와 handler 입니다. 보통 jquery와 같이 함수의 형태로 있는 것을 handler라고 합니다. 이 부분에 대해서는 딱히 설명드릴 부분이 없네요.

```
9   <body>
10    <ul>
11      <li><a href="#">HTML</a></li>
12      <li><a href="#">CSS</a></li>
13      <li><a href="#">JavaScript</a></li>
14    </ul>
15  </body>
```

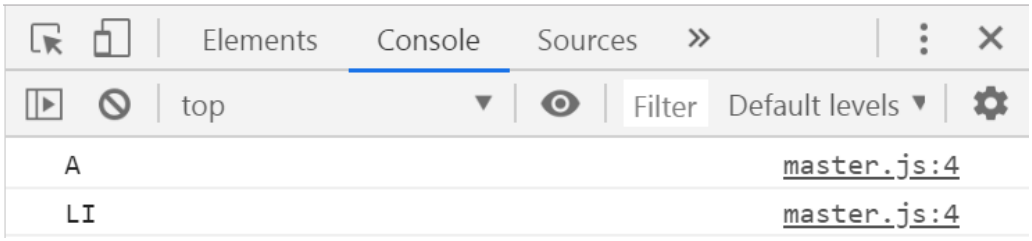
index.html

이 부분은 index.html의 문서 입니다. <ul> 태그 안에 <li>태그와 <a>태그가 있다는 것을 확인해주세요.

```
1  $(document).ready(function(){
2    $('ul').on('click','a, li', function(event){
3      console.log(this.tagName);
4    });
5  });
```

master.js

위의 소스코드에서 가운데에 존재하는 select는 무엇을 의미하냐면 지금 위의 <ul>태그 밑에는 <li> 태그와 <a> 태그가 있습니다. 이 두개의 태그에 대해서 클릭이벤트를 만들겠다는 의미 입니다.



여기서 <a> 태그를 누르면 이렇게 2개의 태그가 실행이되게 됩니다. 왜 ? 버블링으로 인해서 <li>태그에 있는 이벤트가 실행이 되어버리기 때문에 이게 실행되는 것은 그다지 중요한 사실은 아닙니다.

```
9       <ul>
10        <li><a href="#">HTML</a></li>
11        <li><a href="#">CSS</a></li>
12        <li><a href="#">JavaScript</a></li>
13      </ul>
14      <script src="master.js" charset="utf-8"></script>
15    </body>
```

index.html

그동안의 강좌에서는 모듈화라는 것을 하면서 14행의 문구가 <head> 태그에 들어가 있었지만 오늘의 설명에서는 이 렇게 빼야할 이유가 생겨서 아래로 내렸습니다. 이게 아래로 내려가게 되면 master.js 파일도 바뀝니다.

```
1   $('ul').on('click','a, li', function(event){
2       console.log(this.tagName);
3   });
```

master.js

문서 전체를 읽어들여야할 메소드가 사라져도 이는 정상적으로 실행이 됩니다. 그런데 중요한건 이겁니다.

```
8   <body>
9     <script src="master.js" charset="utf-8"></script>
10    <ul>
11      <li><a href="#">HTML</a></li>
12      <li><a href="#">CSS</a></li>
13      <li><a href="#">JavaScript</a></li>
14    </ul>
15  </body>
```

index.html

위의 master.js의 ready가 없애면 이는 실행이 되지 않습니다. 마지막 강좌이기 때문에 여러분들에게 다시 한번 강조하지만 html은 위에서 아래로 컴파일을 하기 때문에 읽을 수 없으면 이는 실행이 되지 않습니다. 이 점은 반드시 여러분들이 잊어버리시면 안됩니다. 그런데 이렇게 master.js를 바꾸면 됩니다.

```
1   $('body').on('click','a, li', function(event){
2       console.log(this.tagName);
3   });
```

master.js

이 부분이 왜 되는지 모르신다면 위에서 제가 html은 어떻게 컴파일이 되는지에 대해 적어두었으니 그 부분을 머릿속에 잘 생각해보시기 바랍니다. 그리고 이 이벤트는 ul 엘리먼트를 포함시키지 않았음에도 불구하고 실행이 된다는 사실 입니다. html은 위에서 아래로 컴파일리 되기 때문에 <ul>, <li>, <a>에 대한 태그를 알 수 없지만 위에서 보여드린 jquery의 on 태그로 인해서 선택된 엘리먼트에 정상적으로 들어갔다는 것이 중요합니다. 등록되지 않은 이벤트에 등록할 수 있는 이 기술의 이름은 late binding라고 합니다. 지금 여러분들에게 알려드린 이 부분은 절대 잊어버리지 마세요. 아마 정말 유용하게 많은 곳에서 쓸 수 있게 됩니다.

```
9     <body>
10      <input type="text" id="target" />
11      <p id="status"></p>
12    </body>
```

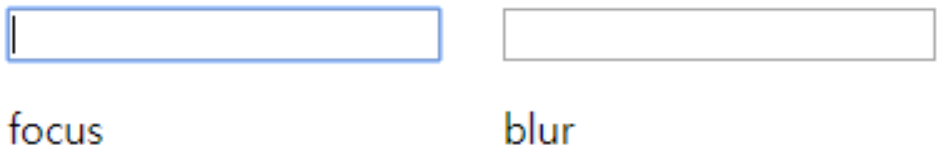
index.html

마지막 사용법은 디자인적 요소를 다양하게 넣고 싶은 분들이 자주 사용할 법한 사용방법 입니다.

```
1  $(document).ready(function(){
2    $('#target').on('focus blur', function(e){
3      $('#status').html(e.type);
4    });
5  });
```

master.js

위에서보면 events 부분에 focus와 blur이 뜨게 됩니다. 이는 <input> 태그의 이벤트 입니다.



같은 기능일 때에는 이렇게 한 줄로 편리하게 사용할 수 있습니다. 다른 기능일 때에는 이렇게 사용하시면 됩니다.

```
$(document).ready(function(){
  $('#target').on({
      'focus' : function(e){
        $('#status').html(e.type);
      },
      'blur' : function(e){
        alert(e.type);
        $('#status').html(e.type);
      }
  });
});
```

master.js

blur에는 다든 이벤트를 넣고 싶다라고 하시면 위의 형식대로 해주시면 되겠습니다. 저는 프로젝트할 때 사실 이런저런 이벤트를 생각해두지 않아서 이 부분은 사용한 적은 없었습니다. 마지막으로 이벤트를 설치할 수 있으면 해제도 할 수 있을 것 입니다. 그런데 사실 이 부분이 크게 쓰일지는 의문 입니다.

```
<body>
  <input type="text" id="target" />
  <input id="remove"  type="button" value="remove" />
  <p id="status"></p>
</body>
```

index.html

이 부분에 대해서는 설명할게 없습니다. id값이 remove인 것에 이벤트를 취소할 예정입니다. 나머지는 위와 똑 같습니다. 그러면 이제 master.js를 보면서 설명을 해드리도록 하겠습니다.

```
1  $(document).ready(function(){
2    var handler = function(e) {
3      $('#status').text(e.type);
4    };
5
6    $('#target').on('focus blur', handler);
7    $('#remove').on('click' , function(e){
8    $('#target').off('focus', handler);
9      console.log(32);
10   });
11 });
```

master.js

2행 부분에 존재하는 변수가 함수라는 것을 제외하고는 하는 일은 똑같습니다. 여기서 중요한 것은 8행 부분에서

off함수를 보시면 됩니다. off 함수의 첫번째 인자값 focus는 종료할 이벤트 입니다. 그리고 2번째 인자값은 함수를 지정하는 부분인데 이 부분에 대해서는 다음걸 보면서 얘기하도록 하겠습니다.

blur

이를 실행하면 이러한 창이 뜨게 됩니다. 처음에는 focus와 blur이 둘다 작동이 되지만 remove를 클릭하면 focus 이벤트는 더이상 작동되지 않는 것을 볼 수 있습니다. 한번 실행해보시면 아 이런거구나 하실겁니다.

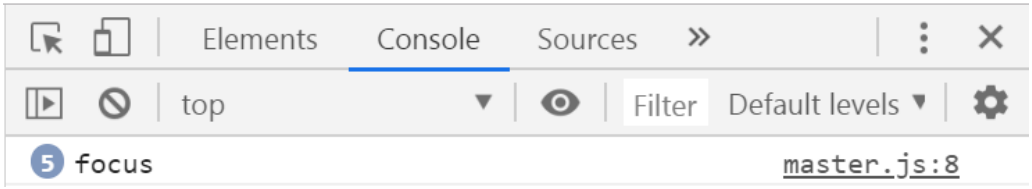
```
1   $(document).ready(function(){
2     var handler = function(e) {
3       $('#status').text(e.type);
4     };
5
6     $('#target').on('focus blur', handler);
7     $('#target').on('focus', function(){
8       console.log('focus');
9     });
10    $('#remove').on('click' , function(e){
11    $('#target').off('focus', handler);
12      console.log(32);
13    });
14  });
```

master.js

위에서 저는 7행 부분을 추가했습니다. 한마디로 event를 하나 더 추가했습니다. 여기서 8행 부분에 alert를 사용하지 않은 이유는 focus는 계속 클릭된 상태 입니다. alert를 사용하게 되면 무한 반복이 일어나게 됩니다.

blur

여기서는 위의 실행과정과 달라진 점은 없습니다. 다만 console창을 보면 focus가 찍히게 됩니다. 그런데 이 상황에서 remove를 클릭하면 삭제되는 것은 handler이라는 함수가 담긴 변수가 적용된 값이 있는 focus 이벤트 뿐 입니다. 이게 아까 위에서 설정한 2번째 인자값 handler이 하는 기능 입니다.



위처럼 console 을 보시면 focus는 계속해서 찍히는 것을 눈으로 확인하실 수 있습니다.

```
11        $('#target').off('focus');
12          console.log(32);
13        });
```

master.js 일부

만약 2개의 이벤트를 전부 종료를 시키고 싶으시면 handler을 제외시키면 focus에 대한 2개의 이벤트가 모두 사라집니다. 이 점도 알아두시면 언젠가 쓰겠죠 뭐...

```
11        $('#target').off('focus blur');
12          console.log(32);
13        });
```

master.js

마지막으로 2개의 이벤트를 모두 종료하고 싶다면 이렇게 설정하면 됩니다. 단, blur을 한번 실행하고 하면 값이 바뀌지 않으니 이를 테스트 할 때엔 remove 버튼을 먼저 클릭하고 실행하시기 바랍니다. 이렇게 해서 jquery의 대한 강좌가 모두 끝이 났습니다. 물론 여러분들이 알아야할 부분들은 많이 있습니다. 또한 이 부분을 공부하면서 소개해드린 생활코딩이라는 사이트에는 3개의 강좌가 더 있습니다. ajax라는 것을 보는 부분인데 사실 이 부분은 서버 프로그래밍을 알고 있지 않으면 이해하기가 힘듭니다. 지금 여러분들에게는 이런게 있다라는 정도 밖에 될 거 같지 않아서 이를 제외하고서 Jquery에 대한 강좌를 마무리 하도록 하겠습니다.

이것으로 [Jquery 강좌] 제 37강 Jquery on 이벤트 사용법 편을
마치도록 하겠습니다.